

目录

1. 测试场景及赛题	2
1.1 测试场景介绍	2
1.2 赛题情况介绍	2
2. 测试环境安装	3
2.1 下载仿真软件	3
2.2 配置运行环境	3
2.3 运行测试任务	4
3. 测试运行说明	4
3.1 场景文件说明	4
3.2 规控算法编写说明	4
4. 评价体系	10

港口集装箱运输作业的高效、稳定运行对提升码头吞吐量与船舶周转效率至关重要。由于港口水平运输系统具有岸桥、堆场等多资源节点强耦合、作业计划动态性强、路径网络复杂等特点，借助自动驾驶技术对运输任务的精准执行能力，实现集卡到达岸桥与堆场的时序精确控制，是保障岸桥和堆场之间的集装箱运输节奏稳定不中断、运输效率提升的核心挑战。

本赛道旨在基于高拟真的港口水平运输作业场景，重点围绕集卡车队运输任务分配、运输路径规划、轨迹级集卡调配（车道引导与速度推荐）等算法开展比赛，解决集卡与集装箱的高效动态匹配、无拥堵路径规划、集卡顶牛冲突化解等问题。

比赛任务涵盖港口场景下的动态任务分派、路径优化与车道级速度引导等环节，要求参赛队伍设计高效可靠的车队调度算法，能够在动态装卸任务与多车资源争抢等条件下，实现运输效率最大化，并通过线上仿真评测与线下专家评审相结合的方式，综合评价算法的实用性、鲁棒性与创新性。

1. 测试场景及赛题

1.1 测试场景介绍

（1）坐标系

对车辆中心点位置 (x, y) 而言， y 轴向下为正向， x 轴向右为正向；对车头朝向 $angle$ 而言， y 轴向上为正向， x 轴向右为正向，角度沿顺时针增加。

（2）单位

测试平台传给选手的数据与选手传给测试平台的数据均使用米制单位。

（3）道路

打开 TESS NG 路网文件后，依次点击“文件” -> “输出路网结构”可以获取路网结构文件。道路分为路段与连接段，当路段 id 与连接段 id 重复时，查看该道路上的车道 id，若车道 id 不为零，该道路的属性为路段；否则该道路的属性为连接段。

（4）车辆

在港区中作业的车辆共有三类，第一类是本公司的自动驾驶集卡，对应的仿真车辆颜色为绿色；第二类是其他公司的自动驾驶集卡，包括自动驾驶的外集卡与其他公司的自动驾驶内集卡，对应的仿真车辆颜色为红色；第三类是人工驾驶集卡，对应的仿真车辆颜色为墨绿色。其中，只有第一类车辆由选手控制。

（5）岸桥与堆场

车辆在岸桥与堆场等待装卸设备完成集装箱的装卸作业，装卸设备距离该路段起点的距离约为 150m。

（6）港区入口与出口

上海外四港中，选手车辆的入口只有一个，对应的路段 id 为 82；出口有两个，对应的路段 id 为 104，191。

1.2 赛题情况介绍

（1）场景任务介绍

测试平台为选手提供多层级的调度手段，包括受控车辆的运输任务分配，时空路径规划以及车速车道引导，要求选手通过精细协调混行环境下的异构行为模式高效完成运输任务。

（2）赛题数量及组成介绍

通过修改港区选址，岸桥与堆场的数量与位置，运输任务的数量、起终点、计划完成时间段，受控车辆的数量与限速，以及港区交通环境，来生成多样化的赛题场景。

共设计三套试卷（A 卷/B 卷/C 卷），总计 140 道赛题(以实际发布为准)，在仿真环境中还原港区作业环境，考察港区智能运输车队调度系统的规控算法能力。

赛题	作用	数量	组成
A 卷	供参赛选手训练并校验，其成绩不参与排名	10	港区选址为上海外四港，供参赛团队训练并校验算法
B 卷	公开考题，供参赛选手提交，其成绩参与排名	80	港区选址为上海外四港、泉州港、罗泾港，供参赛团队提交作品，其成绩参与排名
C 卷	非公开考题，共测试工作人员使用，在赛后公开	50	港区选址为上海外四港、泉州港、罗泾港，作为保留加试题目

2. 测试环境安装

2.1 下载仿真软件

在 [济达交通官网](#) 下载 TESS NG V4.1 版本。

2.2 配置运行环境

为了保证参赛队的规控算法能顺利运行，参赛队应安装统一提供的测试环境，并在之中测试自己的算法，保证其能够顺利运行。

1. 使用 conda 建立虚拟环境，需指定版本为 python3.6.8。

```
conda create -n onsite python=3.6.8
```

2. 激活虚拟环境。

```
conda activate onsite
```

3. 下载 OnSite-Transportation 压缩包，解压至本地文件夹，使用任意 IDE 打开，下载链接：<https://github.com/ykx-51/OnSite-Transportation>。

4. 导入依赖库

```
pip install -r requirements.txt
```

5. 选手还可以参考 [TESS NG 二次开发文档](#) 配置运行环境。

2.3 运行测试任务

完成上述基础配置后，选手即可开始运行测试，检测模块是否正常运行。

在 Windows 环境下运行测试指令：

```
python -u './main.py'
```

如果弹出 TESS NG 仿真页面则代表测试环境安装成功。

在首次运行 TESS NG 时需要导入激活密钥，点击“导入激活码”后选择“同济大学胡箭老师 Onsite 试用.key”激活密钥，提示“激活成功”后关闭程序重新运行即可正常使用。

```
def afterOneStep(self) -> None:
    simu_accuracy: int = self.simuiface.simuAccuracy()
    # 仿真倍速
    simu_multiples: int = self.simuiface.acceMultiples()
    self.simuiface.setAcceMultiples(5) # 设置仿真倍速
```

运行前，可以在“MySimulator.py”的 afterOneStep 中设置仿真倍速，图中仿真倍速为 5。

3. 测试运行说明

3.1 场景文件说明

更换赛题场景时，可能需要替换的文件有：task.csv(task.csv 放在 Output 文件夹中)，vehicle.csv，车道.csv，车道连接.csv，main.py，MySimulator.py。

需要替换的文件会放在文件夹“卷名_场景 id 号_input”中，场景文件下载链接：<https://github.com/ykx-51/OnSite-Transportation>。

3.2 规控算法编写说明

选手可以在测试用例 Baseline.py 中进行算法编写，对测试用例的解析如下：

1. 读取场景信息

```

# 静态输入
input1_1 = pd.read_csv(filepath_or_buffer: "车道.csv", encoding="gbk")
input1_2 = pd.read_csv(filepath_or_buffer: "车道连接.csv", encoding="gbk")
input2 = pd.read_csv(filepath_or_buffer: "Output/task.csv", encoding="gbk")
input2 = input2.dropna()
input3 = pd.read_csv(filepath_or_buffer: "vehicle.csv", encoding="gbk")
# 动态输入
input4 = "Output/vehicle_state.csv"
input5 = "Output/operation_state.csv"
input6 = "Output/id.csv"

# 先删除旧文件
transit.init_output_files()

vehicle_file = None
operation_file = None
id_file = None

vehicle_data = []
operation_data = []
id_data = []

```

```

while True:

    # 读取动态输入的方式，供选手参考
    # ===== 懒加载打开文件 =====
    vehicle_file = open_file_lazy(input4, vehicle_file)
    operation_file = open_file_lazy(input5, operation_file)
    id_file = open_file_lazy(input6, id_file)

    # ===== 读取新数据 =====
    vehicle_file = read_csv_new_lines(vehicle_file, vehicle_data)
    operation_file = read_csv_new_lines(operation_file, operation_data)
    id_file = read_csv_new_lines(id_file, id_data)

```

“task.csv”是受控车辆需要完成的任务信息，车辆在任务的起点与终点均会等待装卸设备完成集装箱装卸作业，每批任务的完成时间不能晚于计划结束时刻。

“vehicle.csv”中存储着受控车辆的数量与限速。

“vehicle_state.csv”用于记录路网中所有车辆在某个仿真时刻的状态，包括该车辆的编号，车型，中心点位置，所处的道路id与车道id，瞬时车速，车头朝向，距离道路起点的距离。每个仿真秒记录一次。

“operation_state.csv”用于记录与受控车辆需要完成的任务相关的装卸设备的作业信息，包括正在等待该装卸设备作业的车辆编号，该装卸设备所处的路段id，该装卸设备的作业状态（0表示空闲，1表示忙碌），本次作业的起始时刻、持续时长与结束时刻。该装卸设备开始工作与结束工作时均会记录一次。

“id.csv”用于记录受控车辆在仿真中的车辆编号，第i行记录选手发出的第i辆车的车辆编号。发车时记录一次。

2. 编写调度算法

（1）分配运输任务

```

# 1.任务分配

# 选手的任务分配算法，选手自己补充；若无，按批次顺序平均分配任务

# 按批次顺序平均分配任务
if vehicle_tasks is None:

    vehicle_num = int(input3["数量"].iloc[0])
    global_start = 82
    global_end = 191

    vehicle_tasks = [[] for _ in range(vehicle_num)]

    # 按批次顺序执行
    for _, row in input2.iterrows():
        quantity = int(row["数量"])
        origin = int(row["起点"])
        destination = int(row["终点"])

        base = quantity // vehicle_num
        remainder = quantity % vehicle_num

        for v in range(vehicle_num):
            task_count = base + (1 if v < remainder else 0)

            for _ in range(task_count):
                vehicle_tasks[v].extend([origin, destination])

    # 加总起点终点
    for v in range(vehicle_num):
        vehicle_tasks[v] = [global_start] + vehicle_tasks[v] + [global_end]

# 输出的车辆任务示例，起点与终点也算任务节点
...

vehicle_tasks = [
    [82, 273, 108, 273, 108, 279, 122, 191], # 第一辆车的任务节点
    [82, 273, 108, 273, 108, 279, 122, 191],
    [82, 273, 108, 273, 108, 279, 122, 191],
    [82, 273, 108, 273, 108, 279, 122, 191],
    [82, 273, 108, 273, 108, 279, 122, 191],
]
...

```

选手可以选择是否编写任务分配算法，若不编写，则按照任务批次的顺序为受控车辆分配任务，分配的原则是尽量使每辆受控车辆需要执行的任务数相同。任务分配算法最后输出的内容为 vehicle_tasks，格式如上图所示。

(2) 规划运输路径

```
# 2.路径规划

# 选手的路径规划算法，选手自己补充；若无，以作业设备间的最短路填充

# 以作业设备间的最短路填充
if vehicle_route is None:
    # 构建 (u, v) -> 最短路 的字典
    shortest_path = pd.read_csv("Shortest path.csv")
    shortest_path_dict = {}

    for _, row in shortest_path.iterrows():
        od = ast.literal_eval(row.iloc[0]) # [273,108]
        path = ast.literal_eval(row.iloc[1]) # [273,88,...,108]
        u, v = int(od[0]), int(od[1])
        shortest_path_dict[(u, v)] = path

# 生成每辆车的全过程路径
vehicle_route = []

for tasks in vehicle_tasks:
    full_route = []

    for i in range(len(tasks) - 1):
        start, end = tasks[i], tasks[i + 1]

        # 取最短路
        path = shortest_path_dict[(start, end)]
```

```
        if i == 0:
            # 第一段路径，完整加入
            full_route.extend(path)
        else:
            # 后续路径，去掉起点，避免重复
            full_route.extend(path[1:])

    vehicle_route.append(full_route)

# 输出的车辆路径示例
...
vehicle_route = [
    [273, ..., 108, ..., 273, ..., 108, ..., 279, ..., 122], # 第一辆车的全过程路径
    [273, ..., 108, ..., 273, ..., 108, ..., 279, ..., 122],
    [273, ..., 108, ..., 273, ..., 108, ..., 279, ..., 122],
]
...
```

选手可以选择是否编写路径规划算法，若不编写，则以任务节点间的最短路径填充车辆路径。路径规划算法最后输出的内容为 vehicle_route，格式如上图所示。其中，Shortest path.csv 通过 Shortest path.py 生成，选手处理从 TESS NG 导出的路网信息时可以参考 Shortest path.py。选手打开 TESS NG 软件后，依次点击“文件”->“输出路网结构”，即可导出路网信息。

选手更新车辆路径时需要注意：新路径中必须包含车辆已走部分的路段 id（车辆当前所在路段也算已走部分），否则 Tess Ng 无法识别车辆的新路径。

（3）引导车速车道

```

# 3.车速车道引导

# 选手的车速车道引导算法，选手自己补充；若无，采用Tess Ng 的模型

# 采用Tess Ng 的模型
# lane_cmd (变道指令, None/1/2), speed_value (None/期望速度的值, m/s)
if vehicle_lane is None:
    num = int(input3.iloc[0, 0])
    vehicle_lane = [[None] for _ in range(num)]
if vehicle_speed is None:
    num = int(input3.iloc[0, 0])
    vehicle_speed = [[None] for _ in range(num)]
# 输出的车辆车道选择示例
'''
vehicle_lane = [
    [None],
    [None],
    [None],
]
'''

```

选手可以选择是否编写车速车道引导算法，若不编写，则采用 Tess Ng 中的模型来引导受控车辆的车速与车道。车速车道引导算法最后输出的内容为 `vehicle_speed` 与 `vehicle_lane`，格式如上图所示。

(4) 更新车辆运输计划

```
# 4. 生成完整的车辆运输计划 + 动态更新
new_plan = []
num = int(input3.iloc[0, 0])

# 输出说明
"""
# 每个元素是 dict, 包含:
# send_time (发送时刻, 单位: s), roadId (起点路段id), state(发送状态, 0/1/2), id (车辆id),
# routing_link_ids (起点到终点的完整路径), routing (当前路段到终点的路径), "_current_idx" (当前在路径中的位置)
# lane_cmd (变道指令, None/1/2), speed_value (期望速度的值, m/s)
"""

for i in range(num):
    vehicle_plan = {
        "send_time": 5 + (i + 1) * 5,
        "roadId": vehicle_route[i][0],
        "state": 0,
        "id": None,
        "routing": None,
        "_current_idx": 0,
        "routing_link_ids": vehicle_route[i],
        "lane_cmd": vehicle_lane[i][0],
        "speed_value": vehicle_speed[i][0],
    }

    new_plan.append(vehicle_plan)

if plan_sent is None:
    update_plan_fn(new_plan)
    plan_sent = True

# 动态更新示例
new_plan[0]["lane_cmd"] = 2 # 引导第一辆车的车道选择
update_plan_fn(new_plan)
```

选手制定车辆运输计划时, 禁止修改“state”、“id”、“routing”、“_current_idx”字段对应的值。“routing_link_ids”包括任务节点所在的路段 id 与其他路段 id, 任务节点所在的路段 id 至少 10min 可更新一次, 其他路段 id 至少 1min 可更新一次, 任务节点所在的路段 id 更新时其他路段 id 也会更新; “speed_value”至少 10s 可更新一次; “lane_cmd”至少 1s 可更新一次。字段更新必须同时满足: 1. Baseline 调用了 update_plan_fn(new_plan); 2. 对应字段距离上次更新时间满足频率限制的要求。

选手第一次生成车辆运输计划时需要生成完整(指包含所有车辆)的运输计划, 后续可对特定车辆的特定字段的值进行动态更新。

3. 生成结果文件

运行完成后, 将 Output 文件夹中的文件打包放入同一个文件夹中, 文件夹名称为场景 id 号。

4. 评测网站

评测网站网址: <https://physiocratic-babara-comely.ngrok-free.dev>。

1 Register

Username:

Email:

Team Name:

Institution:

Mentors (用英文逗号分隔):

Password:

Confirm Password:

Register

Already have an account? [Login](#)

2 Login

Username:

Password:

Login

No account? [Register](#)

3 上传作品

选择卷类别:

A 卷

选择 ZIP 文件:

选择文件

未选择文件

上传并评分

[查看排行榜](#)

4 上传作品

文件上传并评分成功!

选择卷类别:

A 卷

选择 ZIP 文件:

选择文件

未选择文件

上传并评分

下载评分结果

[下载 score.csv](#)

[查看排行榜](#)

5 排行榜

A 卷

账号	测试场景数	总得分
平台测试1	1	-60.0

B 卷

账号	测试场景数	总得分
平台测试2	1	-60.0

C 卷

账号	测试场景数	总得分
平台测试1	1	-60.0
平台测试2	1	-60.0

注册时需要填写的信息中，Username 为本赛道自用排行榜中使用的账号名称，可以匿名；Team Name 为队伍名称；Institution 为学校/组织/单位名称；Mentors 为指导老师姓名列表，若有多位指导老师，指导老师姓名间用英文逗号分隔。Team Name、Institution、Mentors 是 Onsite 平台上显示本赛道排行榜所需要的信息。

ZIP 文件的名称应为“Username_A/B/C”，ZIP 中为多个文件夹，每个文件夹的名称为场景 id 号。

4. 评价体系

排名标准以试题完成率为第一判断指标，试题完成率越高，排名越高（例如 B 卷共 80 套试题，完成试题数目越多，排名越高），在相同完成率的情况下，以完成效果得分进行二次排名，完成效果得分为已完成试题的总得分（例如共 5 题，完成了其中三道题目，每题得分为 80，90，100，则完成效果得分为 270 分）。

单个试题得分标准：

（1）初始分 100 分

（2）评分细则

维度	指标	分值	得分标准
人工成本	顶牛需要人工调解	扣分制	扣分： 受控车辆顶牛次数 x 10
效率	人机冲突	扣分制	扣分： 受控车辆冲突次数 x 2
	单批任务耗时	扣分制	扣分： 超时时长（单位;min） / 2 x 10
	所有任务耗时	得分制	得分： 提前完成的时长（单位;min） / 2 x 10

规范	单批运输任务完成情况	扣分制	若未安排车辆运输该批集装箱，扣 20 分；若某批集装箱中部分任务未完成，按比例扣分。
	作弊	扣分制	丧失参赛资格

（3）检查作弊的方法：选手只能修改 Baseline.py，测试平台会在项目运行前计算其他代码的哈希值；并在项目运行时每隔一段时间计算其他代码的哈希值，防止选手修改其他代码。